PadelMind

AI COACHING FOR RACKET SPORTS

FOR FAMILY EYES — TECHNICAL JUDGEMENT CALL

Technical Briefing — Round 2

Plain-language walk-through of the pipeline, the Ultralytics licensing decision, and four specific questions where a technical opinion would change the plan.

PREPARED FOR

[Son's name]

READING TIME

20–25 minutes

REQUIRED RESPONSE

Four two-liner answers to the questions in §5

AUTHORED

Manoj Maheshwari · 2026-07-03

To: [Son's name] **From:** Papa **Date:** 2026-07-03 **Reading time:** 15 minutes for the briefing, 5 minutes for the licensing question, 5 minutes for the three judgment calls at the end **What I need from you:** Honest technical opinion on **the four questions in Part 5**. Nothing else. No time commitment.

Why this document exists (and why in plain English)

I sent you an earlier document (`SON_TECHNICAL_REVIEW.md`, 2026-07-01) that dived straight into the architecture. Since then I have gone deeper — hardware costs, a 60-day execution plan, and outreach to the OSS author João Silva. In the middle of that, one specific technical question came up that I did not fully understand as a non-technical person: **the Ultralytics / YOLOv8 licensing question**. Half of the technical explanation went over my head — as you'd say, a bouncer.

I would rather ask you now, before we spend money and time in a direction that has a hidden cost we haven't accounted for. So this document has two parts:

1. **A plain-language walk-through** of what the system actually does end-to-end, stage by stage — so you can sanity-check the architecture without hunting through the feasibility report.
2. **The specific licensing question** — laid out in enough detail that you can tell me whether we should pay a US software vendor a recurring fee, or swap to a different model family and pay a one-time engineering cost.

Everything else in the venture is on track. I just need a technical sanity check on this one crossroads before we commit engineering time.

Part 1 — What the system does, in plain English

A player finishes a match at our home club. Within 10 minutes, they receive a WhatsApp message: "You played 47 shots. Your bandeja quality is 3.1 out of 5 — hips underrotated on 60% of them. Watch this drill." Everything in between the match and that WhatsApp message is what the system does.

Here is the flow, in 13 stages, in one sentence each:

#	STAGE	IN PLAIN ENGLISH
1	Camera captures video	A 4K weatherproof IP camera behind the back glass records the full court, all 60 minutes of the match
2	Local recording	A small Raspberry Pi computer in a weatherproof box on court saves the video to a hard drive as it streams — insurance against internet cuts
3	Match window	The Pi detects when the match has actually started and ended (motion + court occupancy), so we upload only the match, not the empty court
4	Upload to cloud	The match video (~3 GB) is uploaded via a 4G/5G cellular modem to Cloudflare storage
5	Wake up the GPU	A tiny cloud function detects the new video and wakes a rented GPU server (RunPod, ~₹2.50/minute)
6	Prep the video	The GPU downloads the video and downsamples it from 4K to 1080p at 30 frames per second (still enough resolution for analysis)
7	Find the players	An AI model called YOLOv8m puts a box around each of the 4 players in every frame and gives each a persistent ID
8	Find each player's skeleton	A second AI model (YOLOv8-pose) draws 13 "joints" (wrists, elbows, shoulders, hips, knees, ankles) on each player — this is what tells us how they move
9	Track the ball	A third AI model (TrackNet) finds the yellow ball in every frame — this is genuinely hard because a padel ball at 100 km/h moves nearly a metre between frames
10	Map to court coordinates	We pre-measured 12 corner points on the court once at install. Every player and ball position gets converted from pixels to metres
11	Detect each shot	When the ball's direction changes sharply, that's a shot. We assign the shot to the nearest player
12	**Classify + score each shot (the moat)**	For each shot, our own trained model looks at the player's skeleton in the second before impact + ball trajectory + court position, and outputs: (a) shot type — bandeja, vibora, smash, etc. from a list of 14, (b) quality 1–5, (c) named flaws — "hips underrotated", "arm angle low"
13	Compose and deliver	We turn all this into a summary — skill score, top-3 weak shots, drill suggestions, 3 highlight clips — and send it via WhatsApp with a link to a phone-friendly web report

Everything from stage 1 through stage 11 is either off-the-shelf hardware or open-source software that we would fork. Stage 12 is the only piece we build ourselves. That is the moat.

The rest of this document is about a licensing question that sits at stages 7 and 8.

Part 2 — The licensing question, laid out plainly

2.1 The situation

Stages 7 and 8 in the table above — the models that find the players and their skeletons — currently use software called **YOLOv8** from a US company called **Ultralytics**. This is the industry-standard model for this kind of task and it's what João's open-source code uses. It is technically excellent.

But it is licensed in a way that matters for us. Ultralytics offers YOLOv8 under two licences:

- **Free** — under a licence called **AGPL-3.0**
- **Paid** — under a commercial "Enterprise Licence" that costs money

We would be running our system on the cloud, and players interact with it over the internet. That "networked service" situation triggers the specific clause in AGPL that says: **if you serve AGPL-licensed code over a network to users, you must open-source your entire application under AGPL to anyone who wants it.**

Our entire application includes the shot classifier that we spend the 60 days building — the moat. If we go free-AGPL, we would legally be required to hand our moat over to anyone who asked, including our future competitors. That is not acceptable.

So the choice for us is:

Option A — Pay Ultralytics. Buy the Enterprise Licence. We do not know the exact price because it is negotiated per company, but public benchmarks put small-startup deals in the range of **$1,000 to $5,000 per year** — call it ₹1 lakh to ₹4 lakh per year. We stay on YOLOv8, keep our moat proprietary, but carry a recurring bill.

Option B — Swap to a different model. There are several models that are technically comparable to YOLOv8 and are released under **truly permissive licences** (Apache-2.0), meaning we owe nothing to anyone. The main candidates are:

- **RTMPose / RTMDet** from a Chinese lab called OpenMMLab (Apache-2.0)
- **RT-DETR** from Baidu (Apache-2.0)
- **YOLOX** from Megvii (Apache-2.0)
- **YOLO-NAS** from Deci (Apache-2.0)

The switch would cost roughly **1 week of a CV engineer's time** to re-tune the padel-specific model weights we already have on the new model architecture. One-time cost of about **₹30,000**. No recurring fee ever.

2.2 The straightforward pros and cons

DIMENSION	OPTION A — PAY ULTRALYTICS	OPTION B — SWAP TO APACHE-2.0 MODEL
Recurring cost	₹1–4 lakh per year forever	Zero
One-time engineering cost	Zero	₹30k (1 engineer-week)
Legal simplicity	Simple — pay invoice, use software	Simple — Apache-2.0 has no obligations
Technical excellence	YOLOv8 is best-in-class, marginal	RTMPose/RT-DETR are within 1–3% accuracy of YOLOv8 on most tasks
Ecosystem support	Massive community, most tutorials use it	Smaller communities but professional, well-documented
Vendor risk	We depend on a US company's future licensing decisions	We depend only on ourselves
João's codebase	Direct fork works with zero changes	We would need to change ~200 lines of pipeline code

2.3 What I have decided provisionally, and why I want your read

My provisional decision is **Option B — swap to RTMPose (or another Apache-2.0 alternative)**. Reasons:

1. It removes a recurring bill from the P&L for the life of the company.
2. It removes a legal dependency on a foreign vendor's future licensing decisions.
3. Even in a worst case where the swap costs us 2–3% accuracy, we can spend the next round of labelled data catching up on our end — accuracy is more about data than model architecture.
4. The 1-week engineering cost is well inside the buffer of the 60-day plan.
5. If it doesn't work out, we can always fall back to paying Ultralytics — no path is closed by trying Option B first.

I want your view because at Apple you deal with model architecture choices at a much higher level of technical judgement than I have. Specifically I want to know if you think there's a subtle reason to stay on YOLOv8 that I'm missing — for example, if the pose keypoint quality of YOLOv8-pose is genuinely superior in a way that would hurt our shot classifier downstream. That is a technical judgement I cannot make on my own.

Part 3 — The 60-day execution plan (very high level)

For context on where this decision sits: we have 60 days to go from hardware-installed at NSCI to a working V1 with real players receiving WhatsApp reports. The full expanded plan is in a companion document (`60_DAY_EXPANDED_PLAN.md`), but at a glance:

- **Days 0–7:** Install cameras and edge computers on two NSCI courts, start capturing matches, coach begins authoring the labelling rubric
- **Days 8–21:** Get the open-source pipeline working end-to-end on our own footage, start labelling clips
- **Days 22–35:** Train the first version of our shot classifier
- **Days 36–50:** Iterate on the classifier, build the player web app and club dashboard
- **Days 51–60:** Wire WhatsApp delivery, coach blind audit, end-to-end demo

The Ultralytics-vs-swap decision needs to be made by roughly **Day 7** so it doesn't disturb the pipeline work. That is why I'm asking you now.

Team is small: me (product), an ML/CV lead (targeting João Silva as consultant or something more), a padel coach (rubric author + labeller), two junior labellers, a full-stack engineer, a frontend engineer, and an AutomationXpert engineer part-time for the WhatsApp side.

Budget for the 60 days is roughly **₹7.2 lakh cash out**, plus a ~₹1 lakh contingency if the classifier plateaus below target accuracy. Hardware itself is about ₹1.2 lakh for both courts including spares.

Part 4 — What I already have as backup

Because I do not want to be dependent on any single OSS repository or vendor, I have taken a full defensive snapshot before making any outreach:

- **João Silva's full `padel_analytics` repo** — mirror-cloned with all 22 refs (main, dev, and all 20 pull-request heads), 24 MB
- **The 4 model weight files** — downloaded from his Google Drive folder, SHA256 verified, 113 MB total
- **6 other reference OSS repos** — TrackNet, TennisCourtDetector, PadelVic dataset, and 3 more, all mirror-cloned
- **Metadata snapshots** — issues, PRs, READMEs captured against the GitHub API on 2026-07-03

Total local archive: **658 MB**. If any of these projects go private, get pulled, or change licence tomorrow, we still have everything at the state it was in on the day we captured it. This is at `/Users/manojmaheshwari/padel-audit/archive/` on my machine.

This means we can proceed with confidence — nothing outside our control can leave us stranded.

Part 5 — The four questions I need your opinion on

Please answer only these. Two-liners are enough. No document work required.

Q1 — On the Ultralytics licensing question:

> *Given the tradeoffs in §2.2, do you agree with my provisional decision to swap to RTMPose / RT-DETR (Apache-2.0) rather than pay for the Ultralytics Enterprise Licence? Is there a technical reason from your Apple work that I'm missing?*

Q2 — On the moat:

> *The moat we're building is the shot classifier at Stage 12. It runs on ~390 features per shot (pose keypoints + ball trajectory + player positions) and outputs 14 shot classes + a 1–5 quality score. The training set will be ~700 coach-labelled clips. In your view, is this the right size problem for a small MLP model, or would you push us toward something more sophisticated (temporal transformer, MotionBERT-style architecture) from day one?*

Q3 — On the edge inference path (Year 2):

> *The plan is to run inference in the cloud (RunPod GPUs) for Year 1, then migrate to edge devices in courts when the fleet exceeds ~200 clubs. The two edge candidates are NVIDIA Jetson Orin (safer, mature CUDA stack) and Apple Mac mini M4 with CoreML + Neural Engine (potentially better performance-per-watt but less community precedent for headless multi-site deployment). This is where your Apple expertise is most valuable. Which path would you prototype first, and why?*

Q4 — On João Silva:

> *João is the author of the base OSS pipeline and I am about to reach out for a 30-minute exploratory call — not to recruit him, just to hear his read on the state of his codebase and what the remaining work looks like. Any signal from you on questions I should ask him, or red flags I should watch for?*

Closing

I know your time is expensive and Apple has first call on your brain. I am not asking you to review architecture drafts weekly or attend design meetings. I am asking for **four short answers**, on a decision that costs me either ₹4 lakh/year forever or ₹30k once. Even directional guidance is worth

more than any of the other technical inputs I have available.

If you'd rather talk than write, a 15-minute call over the weekend works. If you'd rather just answer on WhatsApp in your own words, that also works.

Thank you.

— Papa

Companion documents in this same folder if you want deeper reading: - `TECHNICAL_FEASIBILITY_REPORT.md` — the full architecture and every technical layer (dense) - `60_DAY_EXPANDED_PLAN.md` — the 60-day execution plan with per-workstream detail - `NSCI_PROPOSAL.md` — the proposal we're putting to the club committee - `JOAO_OUTREACH_EMAIL_v2.md` — the humble/exploratory email I'm about to send to João Silva

---

Companion document set: Strategic Venture Plan · Project Report · Technical Feasibility · Commercial Report · Base Club Offer · Technical Review Request.
Single source of truth maintained in `~/Documents/padel-clone/`.